

CONFIG_REFERENCE

HelixCode Notification Configuration Reference

Overview

This document provides a complete reference for configuring the HelixCode notification system, including all channels, reliability features, and integration options.

Configuration Methods

The notification system supports multiple configuration methods:

1. **Programmatic** - Direct configuration in Go code
 2. **Environment Variables** - For sensitive credentials
 3. **YAML Configuration** - For structured settings
 4. **JSON Configuration** - Alternative structured format
-

Notification Channels

Slack Channel

Constructor:

```
channel := notification.NewSlackChannel(webhookURL, channelName,
                                         username)
```

Parameters: - `webhookURL` (string, required): Slack webhook URL from Incoming Webhooks integration - `channelName` (string, optional): Channel to post to (e.g., “#alerts”, “@user”) - `username` (string, optional): Bot display name

Environment Variables:

```
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/YOUR/WEBHOOK/
URL
SLACK_CHANNEL=#alerts
SLACK_USERNAME=HelixBot
```

YAML Configuration:

```
notifications:
  channels:
    slack:
      enabled: true
      webhook_url: ${SLACK_WEBHOOK_URL}
      channel: "#alerts"
      username: "HelixBot"
```

Rate Limit: 1 message per second (Slack API limit)

Discord Channel

Constructor:

```
channel := notification.NewDiscordChannel(webhookURL)
```

Parameters: - webhookURL (string, required): Discord webhook URL from channel integrations

Environment Variables:

```
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/
YOUR_WEBHOOK_ID/YOUR_WEBHOOK_TOKEN
```

YAML Configuration:

```
notifications:
  channels:
    discord:
      enabled: true
      webhook_url: ${DISCORD_WEBHOOK_URL}
```

Rate Limit: 5 messages per 5 seconds (Discord API limit)

Telegram Channel

Constructor:

```
channel := notification.NewTelegramChannel(botToken, chatID)
```

Parameters: - botToken (string, required): Bot token from @BotFather - chatID (string, required): Chat ID to send messages to

Environment Variables:

```
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrsTUVwxyz
TELEGRAM_CHAT_ID=-1001234567890
```

YAML Configuration:

```
notifications:
  channels:
    telegram:
      enabled: true
      bot_token: ${TELEGRAM_BOT_TOKEN}
      chat_id: ${TELEGRAM_CHAT_ID}
```

Rate Limit: 30 messages per second (Telegram API limit)

Getting Chat ID:

```
# Send a message to your bot, then:
curl https://api.telegram.org/bot<TOKEN>/getUpdates
# Look for "chat":{"id": YOUR_CHAT_ID}
```

Email Channel

Constructor:

```
channel := notification.NewEmailChannel(smtpServer, port,
    username, password, fromAddr)
```

Parameters: - smtpServer (string, required): SMTP server address (e.g., "smtp.gmail.com") - port (int, required): SMTP port (587 for TLS, 465 for SSL) - username (string, required): SMTP authentication username - password (string, required): SMTP authentication password - fromAddr (string, required): From email address

Environment Variables:

```
EMAIL_SMTP_SERVER=smtp.gmail.com
EMAIL_SMTP_PORT=587
EMAIL_USERNAME=your-email@gmail.com
EMAIL_PASSWORD=your-app-password
EMAIL_FROM=notifications@yourcompany.com
```

YAML Configuration:

```
notifications:
  channels:
    email:
```

```
enabled: true
smtp_server: ${EMAIL_SMTP_SERVER}
smtp_port: 587
username: ${EMAIL_USERNAME}
password: ${EMAIL_PASSWORD}
from: "notifications@yourcompany.com"
use_tls: true
```

Rate Limit: 10 messages per minute (recommended)

Gmail Configuration: - Enable 2FA - Generate App Password at <https://myaccount.google.com/apppasswords> - Use app password instead of regular password

Generic Webhook Channel

Constructor:

```
headers := map[string]string{
    "Authorization": "Bearer YOUR_TOKEN",
    "X-Custom-Header": "value",
}
channel := notification.NewWebhookChannel(url, headers)
```

Parameters: - url (string, required): Webhook endpoint URL - headers (map[string]string, optional): Custom HTTP headers

Environment Variables:

```
WEBHOOK_URL=https://your-service.com/webhooks/notifications
WEBHOOK_AUTH_TOKEN=your-bearer-token
```

YAML Configuration:

```
notifications:
  channels:
    webhook:
      enabled: true
      url: ${WEBHOOK_URL}
      method: POST
      headers:
        Authorization: "Bearer ${WEBHOOK_AUTH_TOKEN}"
        Content-Type: "application/json"
```

Payload Format:

```
{
  "title": "Notification Title",
  "message": "Notification message body",
  "type": "error",
  "priority": "high",
  "metadata": {
    "custom_field": "value"
  }
}
```

Rate Limit: 100 messages per minute (default)

Microsoft Teams Channel

Constructor:

```
channel := notification.NewTeamsChannel(webhookURL)
```

Parameters: - webhookURL (string, required): Teams incoming webhook URL

Environment Variables:

```
TEAMS_WEBHOOK_URL=https://outlook.office.com/webhook/
YOUR_WEBHOOK_ID
```

YAML Configuration:

```
notifications:
  channels:
    teams:
      enabled: true
      webhook_url: ${TEAMS_WEBHOOK_URL}
```

Creating Webhook: 1. Go to Teams channel 2. Click “...” → Connectors → Incoming Webhook 3. Configure and copy webhook URL

Rate Limit: Same as generic webhook (100/min)

PagerDuty Channel

Constructor:

```
channel := notification.NewPagerDutyChannel(integrationKey)
```

Parameters: - integrationKey (string, required): PagerDuty Events API v2 integration key

Environment Variables:

PAGERDUTY_INTEGRATION_KEY=your-integration-key

YAML Configuration:

```
notifications:
  channels:
    pagerduty:
      enabled: true
      integration_key: ${PAGERDUTY_INTEGRATION_KEY}
```

Getting Integration Key: 1. Go to Services → Your Service → Integrations 2. Add “Events API V2” integration 3. Copy integration key

Severity Mapping: - alert → critical - error → error - warning → warning - info/success → info

Jira Channel

Constructor:

```
channel := notification.NewJiraChannel(baseUrl, email, apiToken,
    projectKey)
```

Parameters: - baseUrl (string, required): Jira instance URL (e.g., “https://yourcompany.atlassian.net”) - email (string, required): Jira account email - apiToken (string, required): Jira API token - projectKey (string, required): Project key (e.g., “HELIX”)

Environment Variables:

JIRA_BASE_URL=https://yourcompany.atlassian.net
JIRA_EMAIL=your-email@company.com
JIRA_API_TOKEN=your-api-token
JIRA_PROJECT_KEY=HELIX

YAML Configuration:

```
notifications:
  channels:
    jira:
      enabled: true
      base_url: ${JIRA_BASE_URL}
      email: ${JIRA_EMAIL}
```

```
api_token: ${JIRA_API_TOKEN}
project_key: ${JIRA_PROJECT_KEY}
```

Creating API Token: 1. Go to <https://id.atlassian.com/manage-profile/security/api-tokens> 2. Create API token 3. Copy and store securely

Priority Mapping: - urgent → Highest - high → High - medium → Medium - low → Low

Issue Type: Always creates as “Task” (can be customized in code)

GitHub Issues Channel

Constructor:

```
channel := notification.NewGitHubIssuesChannel(token, owner,
repo)
```

Parameters: - token (string, required): GitHub personal access token - owner (string, required): Repository owner - repo (string, required): Repository name

Environment Variables:

```
GITHUB_TOKEN=ghp_your_personal_access_token
GITHUB_OWNER=yourcompany
GITHUB_REPO=helixcode
```

YAML Configuration:

```
notifications:
  channels:
    github:
      enabled: true
      token: ${GITHUB_TOKEN}
      owner: ${GITHUB_OWNER}
      repo: ${GITHUB_REPO}
```

Creating Token: 1. Go to Settings → Developer settings → Personal access tokens 2. Generate token with repo scope 3. Copy token

Labels: Automatically adds notification type as label (info, error, warning, etc.)

Retry Configuration

Default Configuration:

```
config := notification.DefaultRetryConfig()  
// Returns:  
// MaxRetries: 3  
// InitialBackoff: 1 second  
// MaxBackoff: 60 seconds  
// BackoffFactor: 2.0 (exponential)  
// Enabled: true
```

Custom Configuration:

```
config := notification.RetryConfig{  
    MaxRetries:      5,  
    InitialBackoff: 2 * time.Second,  
    MaxBackoff:      120 * time.Second,  
    BackoffFactor:   2.5,  
    Enabled:         true,  
}  
retryableChannel :=  
    notification.NewRetryableChannel(baseChannel, config)
```

YAML Configuration:

```
notifications:  
  retry:  
    enabled: true  
    max_retries: 5  
    initial_backoff: 2s  
    max_backoff: 120s  
    backoff_factor: 2.5
```

Backoff Calculation:

$\text{backoff} = \min(\text{initial_backoff} * (\text{backoff_factor} ^ \text{attempt}), \text{max_backoff})$

Examples with defaults:

- Attempt 1: $1s * 2^0 = 1s$
- Attempt 2: $1s * 2^1 = 2s$
- Attempt 3: $1s * 2^2 = 4s$
- Attempt 4: $1s * 2^3 = 8s$

Best Practices: - Enable for all production channels - Use higher max retries for critical notifications - Adjust backoff factor based on API rate limits - Set max backoff to prevent excessive delays

Rate Limiting Configuration

Default Rate Limits:

```
// Predefined limits matching API restrictions
slack:    1 request per second
discord:  5 requests per 5 seconds
telegram: 30 requests per second
email:    10 requests per minute
webhook:  100 requests per minute
```

Getting Default Limiter:

```
limiter := notification.GetDefaultRateLimiter("slack")
rateLimitedChannel :=
    notification.NewRateLimitedChannel(channel, limiter)
```

Custom Rate Limiter:

```
// 10 requests per second
limiter := notification.NewRateLimiter(10, 1*time.Second)
rateLimitedChannel :=
    notification.NewRateLimitedChannel(channel, limiter)
```

YAML Configuration:

```
notifications:
  rate_limiting:
    enabled: true
    limits:
      slack:
        max_requests: 1
        window: 1s
      discord:
        max_requests: 5
        window: 5s
      telegram:
        max_requests: 30
        window: 1s
      email:
        max_requests: 10
        window: 1m
      webhook:
        max_requests: 100
        window: 1m
```

Algorithm: Token bucket with automatic refill

Best Practices: - Always use rate limiting for external APIs - Set limits slightly below API maximums - Monitor rate limit hits in metrics - Combine with queuing for burst handling

Notification Queue Configuration

Creating Queue:

```
queue := notification.NewNotificationQueue(engine, workers,
    maxSize)
queue.Start()
defer queue.Stop()
```

Parameters: - engine (*NotificationEngine, required): Notification engine instance - workers (int, required): Number of concurrent workers - maxSize (int, required): Maximum queue size

YAML Configuration:

```
notifications:
  queue:
    enabled: true
    workers: 5
    max_size: 1000
    processing_timeout: 30s
```

Enqueuing Notifications:

```
err := queue.Enqueue(notification, []string{"slack", "email"},
    maxRetries)
```

Queue Stats:

```
stats := queue.GetStats()
// Returns QueueStats with:
// - Enqueued: Total enqueued
// - Dequeued: Total dequeued
// - Succeeded: Successfully sent
// - Failed: Failed to send
```

Worker Recommendations: - Development: 2-3 workers - Production: 5-10 workers - High volume: 10-20 workers - Adjust based on channel response times

Queue Size Recommendations: - Small deployments: 100-500 - Medium deployments: 500-2000 - Large deployments: 2000-10000

Best Practices: - Use queuing for all async notifications - Monitor queue size to prevent overflow - Adjust workers based on throughput needs - Set appropriate timeouts for processing

Event Bus Configuration

Creating Event Bus:

```
bus := event.NewEventBus()  
bus.SetAsync(true) // Enable async mode
```

YAML Configuration:

```
notifications:  
  events:  
    enabled: true  
    async: true  
    buffer_size: 1000  
    error_logging: true
```

Subscribing to Events:

```
bus.Subscribe(event.EventTaskFailed, func(ctx context.Context,  
    evt event.Event) error {  
    // Handle event  
    return nil  
}))
```

Event Types:

Task Events: - EventTaskCreated - EventTaskAssigned - EventTaskStarted - EventTaskCompleted - EventTaskFailed - EventTaskPaused - EventTaskResumed - EventTaskCancelled

Workflow Events: - EventWorkflowStarted - EventWorkflowCompleted - EventWorkflowFailed - EventStepCompleted - EventStepFailed

Worker Events: - EventWorkerConnected - EventWorkerDisconnected - EventWorkerHealthDegraded - EventWorkerHeartbeatMissed

System Events: - EventSystemStartup - EventSystemShutdown - EventSystemError

Best Practices: - Use async mode for better performance - Subscribe only to needed event types - Handle errors in event handlers - Use global bus for system-wide events

Notification Rules Configuration

Adding Rules:

```
engine.AddRule(notification.NotificationRule{
    Name:      "Critical Errors",
    Condition:  "type==error && priority==urgent",
    Channels:   []string{"slack", "pagerduty", "email"},
    Priority:   notification.NotificationPriorityUrgent,
    Enabled:    true,
})
```

YAML Configuration:

```
notifications:
  rules:
    - name: "Critical Errors"
      condition: "type==error && priority==urgent"
      channels: ["slack", "pagerduty", "email"]
      priority: "urgent"
      enabled: true

    - name: "Task Failures"
      condition: "type==error && source==task_manager"
      channels: ["slack"]
      priority: "high"
      enabled: true

    - name: "Worker Health Issues"
      condition: "type==warning && source==worker_pool"
      channels: ["slack", "email"]
      priority: "medium"
      enabled: true

    - name: "Build Successes"
      condition: "type==success && source==build"
      channels: ["discord"]
      priority: "low"
      enabled: true
```

Condition Syntax: - type==error: Match notification type - priority==urgent: Match priority - source==task_manager: Match event source - Combine with && (AND), || (OR)

Priority Levels: - urgent: Critical, immediate attention - high: Important, timely attention - medium: Normal priority - low: Informational

Metrics Configuration

Creating Metrics:

```
metrics := notification.NewMetrics()
```

Recording Metrics:

```
metrics.RecordSent("slack", duration)
metrics.RecordFailed("email")
metrics.RecordRetry("discord")
metrics.RecordQueued()
metrics.RecordRateLimited("telegram")
metrics.RecordEventProcessed()
metrics.RecordEventIgnored()
```

Getting Metrics:

```
current := metrics.GetMetrics()
successRate := metrics.GetSuccessRate()
channelRate := metrics.GetChannelSuccessRate("slack")
```

YAML Configuration:

```
notifications:
  metrics:
    enabled: true
    retention_period: 24h
    export_interval: 1m
    exporters:
      - type: "prometheus"
        endpoint: ":9090/metrics"
      - type: "statsd"
        host: "localhost:8125"
```

Available Metrics: - TotalSent: Total notifications sent - TotalFailed: Total failures - TotalRetries: Total retry attempts - TotalQueued: Total queued notifications - TotalRateLimited: Total rate-limited - AverageResponseTime:

Average send time - MinResponseTime: Fastest send - MaxResponseTime: Slowest send - EventsProcessed: Events processed - EventsIgnored: Events ignored

Per-Channel Metrics: - Sent: Notifications sent - Failed: Failures - Retries: Retry attempts - RateLimited: Rate limit hits - AvgTime: Average response time

Complete Configuration Example

YAML (config.yaml):

```
notifications:
  # Channels
  channels:
    slack:
      enabled: true
      webhook_url: ${SLACK_WEBHOOK_URL}
      channel: "#alerts"
      username: "HelixBot"

    discord:
      enabled: true
      webhook_url: ${DISCORD_WEBHOOK_URL}

    telegram:
      enabled: true
      bot_token: ${TELEGRAM_BOT_TOKEN}
      chat_id: ${TELEGRAM_CHAT_ID}

    email:
      enabled: true
      smtp_server: ${EMAIL_SMTP_SERVER}
      smtp_port: 587
      username: ${EMAIL_USERNAME}
      password: ${EMAIL_PASSWORD}
      from: "notifications@company.com"
      use_tls: true

    pagerduty:
      enabled: true
      integration_key: ${PAGERDUTY_INTEGRATION_KEY}

    jira:
      enabled: true
      base_url: ${JIRA_BASE_URL}
```

```
    email: ${JIRA_EMAIL}
    api_token: ${JIRA_API_TOKEN}
    project_key: "HELIX"

github:
  enabled: false
  token: ${GITHUB_TOKEN}
  owner: "yourcompany"
  repo: "helixcode"

# Retry configuration
retry:
  enabled: true
  max_retries: 3
  initial_backoff: 1s
  max_backoff: 60s
  backoff_factor: 2.0

# Rate limiting
rate_limiting:
  enabled: true
  limits:
    slack:
      max_requests: 1
      window: 1s
    discord:
      max_requests: 5
      window: 5s
    telegram:
      max_requests: 30
      window: 1s
    email:
      max_requests: 10
      window: 1m

# Queue configuration
queue:
  enabled: true
  workers: 5
  max_size: 1000
  processing_timeout: 30s

# Event bus
events:
```

```
enabled: true
async: true
buffer_size: 1000

# Notification rules
rules:
  - name: "Critical Errors"
    condition: "type==error && priority==urgent"
    channels: ["slack", "pagerduty", "email"]
    priority: "urgent"
    enabled: true

  - name: "Task Failures"
    condition: "type==error"
    channels: ["slack"]
    priority: "high"
    enabled: true

  - name: "Worker Issues"
    condition: "type==warning"
    channels: ["slack"]
    priority: "medium"
    enabled: true

# Metrics
metrics:
  enabled: true
  retention_period: 24h
```

Environment Variables (.env):

```
# Slack
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/YOUR/WEBHOOK/
URL

# Discord
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/ID/TOKEN

# Telegram
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrsTUVwxyz
TELEGRAM_CHAT_ID=-1001234567890

# Email
EMAIL_SMTP_SERVER=smtp.gmail.com
EMAIL_SMTP_PORT=587
```

```
EMAIL_USERNAME=your-email@gmail.com
EMAIL_PASSWORD=your-app-password
```

```
# PagerDuty
```

```
PAGERDUTY_INTEGRATION_KEY=your-integration-key
```

```
# Jira
```

```
JIRA_BASE_URL=https://yourcompany.atlassian.net
```

```
JIRA_EMAIL=your-email@company.com
```

```
JIRA_API_TOKEN=your-api-token
```

```
# GitHub
```

```
GITHUB_TOKEN=ghp_your_token
```

Security Best Practices

1. **Never commit credentials** - Use environment variables
 2. **Use app-specific passwords** - For email services
 3. **Rotate tokens regularly** - Especially API tokens
 4. **Restrict permissions** - Use minimum required scopes
 5. **Use HTTPS** - For all webhook URLs
 6. **Validate webhook sources** - Check webhook signatures
 7. **Encrypt sensitive data** - In configuration files
 8. **Use secrets management** - HashiCorp Vault, AWS Secrets Manager
 9. **Monitor access logs** - Track notification usage
 10. **Implement rate limiting** - Prevent abuse
-

Troubleshooting

Channel Not Sending

Check: 1. Channel enabled: `channel.IsEnabled()` returns true 2. Valid credentials: No masked values in production 3. Network connectivity: Can reach API endpoints 4. Rate limits: Check metrics for rate limit hits 5. Error logs: Review error messages

Rate Limiting Issues

Solutions: 1. Increase window duration 2. Decrease max requests 3. Enable queuing to buffer bursts 4. Use multiple channels for high volume

Queue Overflow

Solutions: 1. Increase queue size 2. Add more workers 3. Optimize channel response times 4. Implement priority queuing

Slow Notifications

Check: 1. Network latency to APIs 2. Number of retries configured 3. Worker pool size 4. Channel response times in metrics

Performance Tuning

High Volume Scenarios

Configuration:

```
notifications:
  queue:
    workers: 20
    max_size: 10000

  rate_limiting:
    enabled: true
    # Use burst-friendly limits

  retry:
    max_retries: 2 # Reduce for faster failure
    initial_backoff: 500ms
```

Low Latency Requirements

Configuration:

```
notifications:
  queue:
    workers: 10
    processing_timeout: 5s

  retry:
    enabled: false # Direct send, no retries

  rate_limiting:
    enabled: false # Remove rate limiting overhead
```

Reliability Priority

Configuration:

```
notifications:
  queue:
    workers: 5
    max_size: 5000

  retry:
    max_retries: 5
    max_backoff: 300s

rate_limiting:
  enabled: true
```

Monitoring Recommendations

Key Metrics to Monitor: 1. Success rate (target: >99%) 2. Average response time (target: <1s) 3. Queue size (alert if >80% full) 4. Rate limit hits (alert if frequent) 5. Retry rate (alert if >10%) 6. Channel-specific success rates

Alerting Rules:

```
alerts:
- name: "Low Success Rate"
  condition: success_rate < 95%
  severity: warning

- name: "Queue Near Full"
  condition: queue_size > 800
  severity: warning

- name: "High Retry Rate"
  condition: retry_rate > 15%
  severity: warning
```

For more information, see: - [API Reference](#) - [Testing Guide](#) - [Event Integration Guide](#)